NeuroVision-Pi4 | Full Recovery & Rebuild Manual

Revision: 22 October 2026 | Source: live GitHub repository drpeker/NeuroVision-Pi4, default branch neurovision-raspi-4. This document is written in English for reproducible global development.

Scope: Raspberry Pi 4 (neurobrain), XiaoZhi voice assistant, Logitech C270 camera, SSD MobileNet v2 vision service, MJPEG, and MCP get_visible_objects. The separate Raspberry Pi Zero 2 W (neuroeye) and offline llama.cpp brain are NOT automatically restored by this repository.

1. Recovery assurance and limitations

VERIFIED: code paths, camera configuration, startup script, MCP registration, and declared project dependencies were inspected directly from GitHub. NOT VERIFIED: actual machine package versions, hidden credentials, audio device indexes, exact Python venv lock state, OS image, network settings, model binary integrity, and current working state. Do not mistake repository contents for a complete disk image.

IMPORTANT: GitHub read access succeeded, but the attempted README update returned HTTP 403 (integration cannot write). This PDF does not modify the repository.

2. Known machine and architecture

Host: neurobrain; user: drpeker; board: Raspberry Pi 4 Model B, 4 GB RAM; OS reported Debian 13 Trixie, ARM64; USB Logitech C270 camera/microphone; analog audio and external amplifier. Application root: /home/drpeker/projects/neurovision. Startup entry: ~/.local/bin/neuro-vision.

Pipeline: C270 -> OpenCV V4L2 -> RGB/PIL -> LiteRT SSD MobileNet v2 -> /detections JSON and /stream MJPEG -> MCP get_visible_objects -> XiaoZhi voice AI -> Turkish spoken output. The voice path depends on separately provisioned activation credentials and audio configuration.

3. Before wiping anything: capture a real recovery snapshot

Run these on the EXISTING Pi before reinstalling. They collect versions and configuration evidence without exposing secrets in public repositories. Review the resulting archive locally.

```
mkdir -p ~/neurovision-recovery
cd ~/neurovision-recovery
date -Is > timestamp.txt
uname -a > kernel.txt
cat /etc/os-release > os-release.txt
python3 --version > python.txt 2>&1
dpkg-query -W > dpkg-packages.txt
python3 -m pip freeze > system-pip-freeze.txt 2>&1
lsusb > usb.txt
v4l2-ctl --list-devices > video-devices.txt 2>&1
arecord -l > audio-inputs.txt 2>&1
aplay -l > audio-outputs.txt 2>&1
ip -brief address > ip-addresses.txt
systemctl --type=service --state=running > services.txt
df -h > disks.txt
lsblk -f > partitions.txt
cd ~/projects/neurovision
git remote -v > ~/neurovision-recovery/git-remotes.txt
git status --short > ~/neurovision-recovery/git-status.txt
git rev-parse HEAD > ~/neurovision-recovery/git-commit.txt
.venv/bin/python -m pip freeze > ~/neurovision-recovery/venv-freeze.txt 2>&1
readlink -f ~/.local/bin/neuro-vision > ~/neurovision-recovery/launcher.txt
```

Also securely back up user-specific configuration, activation data, device registration, .env files, audio preferences, downloaded model binaries, and local changes NOT committed to Git. Do not upload credentials

or tokens to a public repository. Prefer an offline encrypted backup or a full SD/SSD image.

4. Rebuild operating system and connectivity

Flash a compatible Raspberry Pi OS/Debian ARM64 image using Raspberry Pi Imager. The original system was reported as Debian 13 Trixie; if using another image, dependency behavior may differ. Set hostname neurobrain, user drpeker, SSH access, locale, timezone, and Wi-Fi/Ethernet. Do not assume password or IP address.

```
hostnamectl
cat /etc/os-release
uname -m
sudo systemctl enable --now ssh
ssh drpeker@neurobrain.local
sudo apt update
sudo apt install -y git curl ca-certificates build-essential \
  python3 python3-venv python3-pip python3-dev \
  ffmpeg v4l-utils alsa-utils libportaudio2 portaudio19-dev \
  libsndfile1 libopus0 libgl1 libglib2.0-0
```

The apt package list is a reasonable bootstrap, not an exact captured manifest. Compare with dpkg-packages.txt from the old installation; repository dependencies may need additional libraries.

5. Restore repository: correct branch, not upstream XiaoZhi

```
mkdir -p ~/projects
cd ~/projects
git clone --branch neurovision-raspi-4 \
  https://github.com/drpeker/NeuroVision-Pi4.git neurovision
cd ~/projects/neurovision
git branch --show-current
git log -1 --oneline
git status --short
```

Do NOT replace this fork with a fresh clone of huangjunsen0406/py-xiaozhi: the fork contains the NeuroVision-specific MCP registration and vision service. Preserve the exact commit hash from the original machine when available.

6. Python environment: reproducible procedure

The upstream README-XIAOZHI.md states Python 3.10–3.12; the project pyproject.toml specifies >=3.10. For a conservative recovery use Python 3.12 if available, not an untested Python 3.13 environment. The repository recommends uv sync; use a lockfile if the repository contains one.

```
cd ~/projects/neurovision
python3 --version
command -v uv || echo "Install uv from its official instructions"
uv python install 3.12
uv sync --python 3.12
uv run python -c "import cv2, PIL, numpy; print('core imports OK')"
```

CRITICAL: the vision code imports ai_edge_litert.interpreter.Interpreter. The inspected pyproject.toml and requirements.txt do NOT explicitly declare ai-edge-litert. Install an ARM64-compatible LiteRT package only after verifying available wheel support and record its exact version. Do not blindly use an incompatible x86 wheel.

```
cd ~/projects/neurovision
.venv/bin/python -c \
  "from ai_edge_litert.interpreter import Interpreter; print('LiteRT OK')"
.venv/bin/python -c "import cv2; print(cv2.__version__)"
```

If uv is unavailable, create a Python 3.12 virtual environment and install requirements after checking platform constraints. The pip and uv resolution paths are not necessarily identical.

7. Vision model files: mandatory

The vision service uses EXACT relative paths vision/models/ssd_mobilenet_v2.tflite and vision/models/coco_labels.txt. Both must exist; the model binary cannot be regenerated from labels alone. Copy any missing .tflite binary from your verified backup or a validated original release.

```
cd ~/projects/neurovision
ls -lh vision/models/ssd_mobilenet_v2.tflite \
       vision/models/coco_labels.txt
sha256sum vision/models/ssd_mobilenet_v2.tflite
head -10 vision/models/coco_labels.txt
```

Labels are zero-based COCO-style mappings in the inspected file. Preserve matching class ordering and output tensor interpretation. Do not substitute a different TFLite model without verifying its input shape, tensor ordering, and labels.

8. Camera capture and model inference: exact code settings

Source: vision/neurovision_live.py. Camera cv2.VideoCapture(0, cv2.CAP_V4L2); FOURCC MJPG; requested width 640; height 480; FPS 30. Model threshold 0.40; LiteRT interpreter num_threads=2. Captured BGR frames are converted to RGB; PIL resizes to the model input dimensions; floating-point inputs are normalized with (x - 127.5) / 127.5. Output tensors are assumed boxes, classes, scores, count, in that order.

JPEG output uses quality=70. HTTP server binds 0.0.0.0:8080 and serves /, /stream, /detections. MJPEG sends a frame approximately every 0.2 s to each client, which is NOT equivalent to guaranteed 30 FPS inference or stream delivery.

```
sudo apt install -y v4l-utils
v4l2-ctl --list-devices
v4l2-ctl -d /dev/video0 --list-formats-ext
ls -l /dev/video*
groups
cd ~/projects/neurovision
.venv/bin/python vision/neurovision_live.py
```

If the camera is not /dev/video0, inspect the V4L2 device list and change the index only after confirming the correct node. USB video device numbering can change between boots. If access is denied, inspect permissions and group membership (often video).

9. Camera endpoint validation

```
curl -fsS http://127.0.0.1:8080/detections
curl -I http://127.0.0.1:8080/
ss -ltnp | grep ':8080'
# Browser on local network:
# http://neurobrain.local:8080/
# http://neurobrain.local:8080/stream
# http://neurobrain.local:8080/detections
```

The detection endpoint can return [] when no object is detected; this alone is not an error. /stream is multipart MJPEG, not a single JPEG. The server currently has no authentication and binds all interfaces; use only on a trusted LAN or add access controls before exposure.

10. MCP integration: exact source and wiring

File src/mcp/tools/neurovision/register.py registers an MCP tool named get_visible_objects with an empty PropertyList. It calls http://127.0.0.1:8080/detections using urllib.request.urlopen with timeout=3 seconds and returns JSON containing source, objects, and a Turkish description instruction. If unavailable, it returns an error JSON payload.

File src/mcp/mcp_server.py, method add_common_tools, imports register_neurovision_tools from src.mcp.tools.neurovision and invokes register_neurovision_tools(self.add_tool). The src/mcp/tools/neurovision/__init__.py re-exports that function. This explicit registration is a CRITICAL fork-specific modification.

```
grep -n 'register_neurovision_tools' \
  src/mcp/mcp_server.py src/mcp/tools/neurovision/*.py
grep -n 'get_visible_objects\|127.0.0.1:8080' \
  src/mcp/tools/neurovision/register.py
grep -R -n 'MCP_TOOLS' . --exclude-dir=.git | head
```

If MCP_TOOLS.DISABLED contains get_visible_objects, it will not be advertised. The server uses JSON-RPC 2.0, supports initialize, tools/list, tools/call, and returns paginated tool lists. Do not assume an MCP tool is working merely because the camera webpage opens: test an actual tool call through the XiaoZhi protocol.

11. Startup script and exact runtime order

Source: scripts/neuro-vision. The script changes to /home/drpeker/projects/neurovision, selects $PWD/.venv/bin/python, checks /detections on port 8080, launches the vision service with setsid, waits up to 40 x 0.5 s (roughly 20 seconds plus HTTP overhead), then starts main.py --mode cli. Ctrl+C should trigger cleanup of the camera process group.

```
cd ~/projects/neurovision
chmod +x scripts/neuro-vision
mkdir -p ~/.local/bin
ln -sfn "$HOME/projects/neurovision/scripts/neuro-vision" \
  "$HOME/.local/bin/neuro-vision"
~/.local/bin/neuro-vision
```

If port 8080 is occupied, the script deliberately refuses to launch. Do not kill unrelated processes without checking ownership. If .venv/bin/python does not exist, rebuild the virtual environment first. Do not run two independent copies of the vision server.

12. XiaoZhi activation, audio and network

The upstream application uses online WebSocket/MQTT services and device authentication. The repository does not reveal the user-specific activation credentials or currently selected server configuration. Restore these from a secure backup or re-register the device through the service. Do not fabricate activation tokens or hardcode secrets in README files.

```
arecord -l
aplay -l
python3 -c "import sounddevice as sd; print(sd.query_devices())"
cd ~/projects/neurovision
.venv/bin/python main.py --mode cli
```

Verify that the selected microphone is actually the Logitech C270 or intended USB microphone, and that playback uses the desired analog audio device/amplifier. Test capture and playback separately before debugging network AI responses. Check ALSA/PulseAudio/PipeWire device routing and volume; these were not independently captured.

13. Failure diagnostics: symptoms and checks

Camera unavailable: check lsusb, v4l2-ctl, /dev/video*, permissions, camera index, power and USB cable. LiteRT import error: verify architecture, wheel and venv interpreter. Empty detections: verify actual frames, lighting, 0.40 threshold, matching labels and tensor ordering. Wrong colors: ensure BGR-to-RGB conversion remains intact.

MCP tool missing: check register_neurovision_tools in mcp_server.py, __init__.py export, and disabled-tool configuration. MCP errors: curl localhost:8080/detections from the SAME Pi, then inspect timeout or server logs. No XiaoZhi voice: test audio I/O, service registration, authentication, network, and server logs. Port collision: ss -ltnp | grep 8080.

```
ps aux | grep -E 'neurovision_live|main.py' | grep -v grep
ss -ltnp | grep 8080
free -h
dmesg -T | tail -60
journalctl -b --no-pager -n 100
git diff --stat
git status --short
```

14. Backups, security and long-term reproducibility

Create a tagged known-good commit after testing. Save SHA256 checksums for model binaries, a dependency lock, hardware inventory, sanitized configuration template, and recovery test results. Back up actual credentials separately and encrypted. Use git status before every destructive command. A Git clone restores tracked source code, NOT your device registration, .venv, model files omitted by Git, OS services, or local settings.

```
cd ~/projects/neurovision
git status --short
git rev-parse HEAD
sha256sum vision/models/*.tflite > ~/neurovision-recovery/models.sha256
tar -czf ~/neurovision-source-snapshot.tar.gz \
  --exclude=.git --exclude=.venv -C ~/projects neurovision
```

A full block-device image is the only reliable way to preserve unknown system-level customizations exactly. Verify backups by restoring onto a spare SD/SSD before erasing the working system.

15. Final acceptance checklist

[ ] Host neurobrain resolves over LAN; SSH login works for drpeker.

[ ] Correct fork branch and commit checked out; no unintended uncommitted changes.

[ ] Python venv exists and imports cv2, PIL, numpy, and LiteRT.

[ ] SSD MobileNet v2 binary and coco_labels.txt exist and are compatible.

[ ] Logitech C270 detected; frames read at a verified negotiated format.

[ ] HTTP /, /stream and /detections respond locally and from intended LAN client.

[ ] get_visible_objects is advertised and returns detection JSON through MCP.

[ ] Microphone capture, speaker output, and XiaoZhi activation independently pass.

[ ] Launcher starts both processes and Ctrl+C terminates them cleanly.

[ ] Offline encrypted backup and validated disk image exist.

16. Source inventory and important distinction

GitHub repository: https://github.com/drpeker/NeuroVision-Pi4 ; branch neurovision-raspi-4. Files directly inspected: README.md; README-XIAOZHI.md; scripts/neuro-vision; vision/neurovision_live.py; vision/models/coco_labels.txt; src/mcp/tools/neurovision/register.py; src/mcp/tools/neurovision/__init__.py; src/mcp/mcp_server.py; pyproject.toml; requirements.txt.

Other remembered projects are distinct: Raspberry Pi Zero 2 W neuroeye has its own NeuroEye repository and CSI OV5647 camera; the earlier offline Raspberry Pi 4 Qwen/Whisper/Piper stack has different startup and model paths. Neither should be silently merged into this XiaoZhi recovery procedure. Their complete rebuild requires their respective source and current device inventories.

Appendix A — Verified Live-System Baseline (23 October 2026)

This appendix updates the earlier recovery manual using terminal outputs supplied from the working Raspberry Pi. It supersedes any conflicting assumptions in the original document. It is not a disk image, and it does not contain credentials, runtime configuration secrets, or a verified complete package lockfile.

Host, operating system, hardware

Field	Verified value
Hostname / user	neurobrain / drpeker
OS	Debian GNU/Linux 13.7 (trixie)
Kernel	6.18.50+rpt-rpi-v8; aarch64
System Python	3.13.5
Repository	/home/drpeker/projects/neurovision
Git HEAD	da9539b9cb542cae6eb44ea823afde00c97be997
Git working tree	No changes reported by git status --short
Memory / swap	3.7 GiB RAM; 2.0 GiB swap
Root storage	/dev/mmcblk0p2; 58 GB filesystem, 19 GB used, 37 GB available
Camera	Logitech C270; USB 046d:0825; /dev/video0
Capture configuration	640 x 480, MJPG, 30/1 FPS
Audio capture (ALSA)	C270 HD WEBCAM: card 3, device 0
Audio playback (ALSA)	bcm2835 Headphones: card 0, device 0; HDMI cards 1 and 2

SoundDevice observation and limitation

The live sounddevice query returned output devices 0 (bcm2835 Headphones), 1 (sysdefault), and 2 (default), with sd.default.device = [-1, 2]. No capture device appeared in that query, although ALSA listed the C270 capture device. The voice application was running during inspection. Do not conclude that the microphone is defective, or change the working audio configuration on the basis of this snapshot alone. Device indices can change after reboot.

Verified Python packages (selected exact versions)

Package	Version
ai-edge-litert	2.3.0
numpy	2.4.0
opencv-python-headless	4.11.0.86
pillow	12.1.0
sherpa-onnx / sherpa-onnx-core	1.13.0 / 1.13.0
sounddevice	0.5.3
opuslib	3.0.1
soxr	1.0.0
aiohttp	3.13.2
websockets	15.0.1

Package	Version
openai	2.14.0
py-xiaozhi	1.0.0, editable at /home/drpeker/projects/neurovision

The full package listing was shown in the session, but the on-device freeze file has not been confirmed as created. Python 3.13.5 is the observed working interpreter; do not blindly replace it with 3.12. The virtual environment lacks the pip module, which is not inherently a failure when uv manages dependencies.

Minimal read-only preservation commands

Run from the Pi when convenient. These commands write only inventory files to ~/neurovision-recovery and do not modify application configuration:

```
mkdir -p ~/neurovision-recovery

uv pip freeze --python ~/projects/neurovision/.venv/bin/python > ~/neurovision-recovery/python-packages.lock.txt

git -C ~/projects/neurovision rev-parse HEAD > ~/neurovision-recovery/git-commit.txt
```

Verify the freeze file exists and has entries:

```
wc -l ~/neurovision-recovery/python-packages.lock.txt
```

For true reproducibility, also preserve the Git-tracked source and model binaries, any uv.lock if present, and required private configuration separately. Never commit API keys, tokens, device credentials, or private identifiers to a public repository.

Recovery priority and acceptance checks

1. Rebuild Debian 13 ARM64, recover repository at the recorded commit, and restore the compatible Python 3.13 environment using uv and the package snapshot. 2. Verify model files and local vision endpoint at http://127.0.0.1:8080/detections. 3. Verify the XiaoZhi CLI starts through scripts/neuro-vision and that the MCP tool get_visible_objects responds. 4. Confirm microphone and analog output with actual application tests, without relying on persistent ALSA/PortAudio numeric indices. 5. Restore any private service configuration from a secure backup.

Status: Documentation baseline complete. A guaranteed one-command restoration still requires separately archived secrets/configuration, model binaries, and the confirmed full freeze file.